

Instructions

RS and RX formats

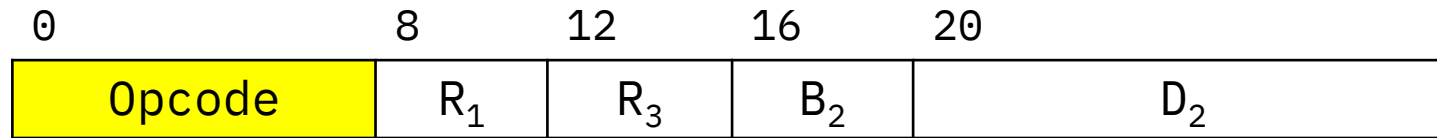
Instruction formats

- Next, RS instructions

Format	Description
Register-and-register (RR)	Both operands are registers
Register-and-storage (RS)	One operand is a register
Register-and-indexed-storage (RX)	and another a storage address
Storage-and-immediate (SI)	One operand is a storage address and another an “immediate” value
Storage-and-storage (SS)	Both operands are storage addresses
Register-and-storage (RXY)	One operand is a register and another a long displacement address
Register-and-immediate (RI)	One operand is a register and another a 16-bit relative address

RS instructions

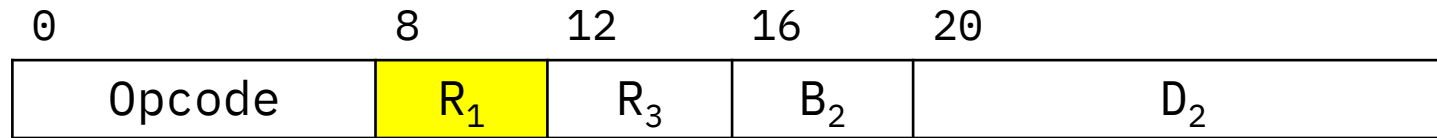
- RS instructions occupy 2 halfwords
- In memory, they look like this:



Operation code

RS instructions

- RS instructions occupy 2 halfwords
- In memory, they look like this:

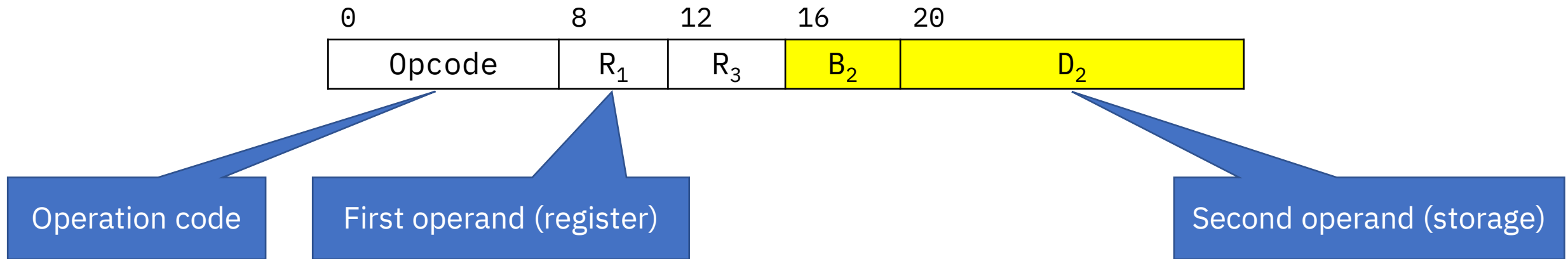


Operation code

First operand (register)

RS instructions

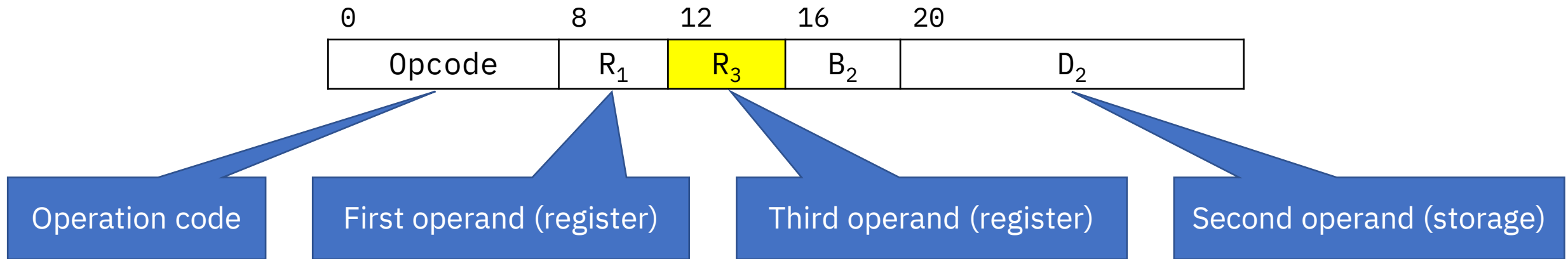
- RS instructions occupy 2 halfwords
- In memory, they look like this:



- The second operand is a 12-bit base-displacement halfword

RS instructions

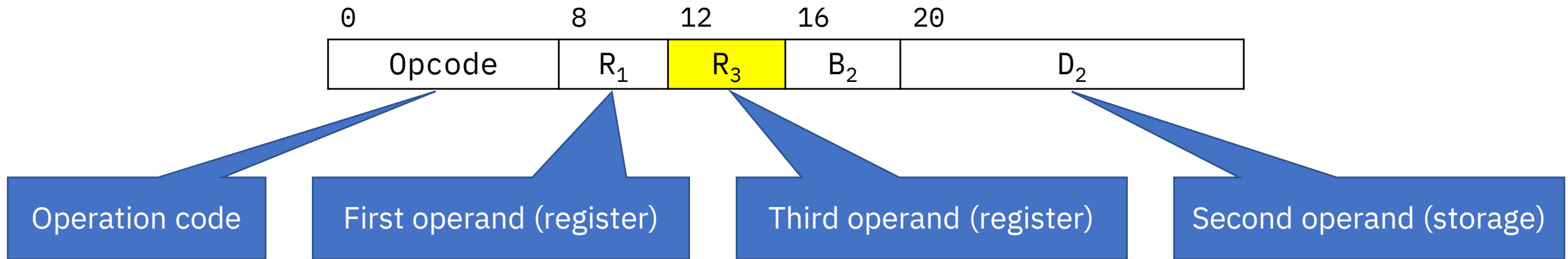
- RS instructions occupy 2 halfwords
- In memory, they look like this:



- The second operand is a 12-bit base-displacement halfword

RS instructions

- RS instructions occupy 2 halfwords
- In memory, they look like this:



- The second operand is a 12-bit base-displacement halfword
- Later, we'll see RS instructions where the third operand is not a register

RS instructions - addressing halfword

- Before we look at the example, we need to know how to specify the base-displacement operand in Assembler
- We saw that registers are easy; we refer to them by their number
- The Assembler syntax for a base-displacement operand is:

`displacement(base register)`

RS instructions - addressing halfword

- For example, 255 bytes from the address at register 10, written in Assembler, is:

255(10)

- The resulting addressing halfword, in memory, is:

0	3	4	15
A	0	F	F

- Now, the promised example

RS instructions - example

- Our RS instruction example is LOAD MULTIPLE
 - It loads consecutive registers from consecutive fullwords in memory
- Its Assembler mnemonic is **LM**
- Its opcode is **98**

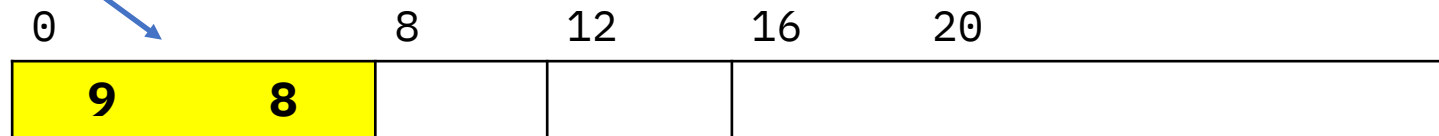
RS instructions - LOAD MULTIPLE example

- We want to load registers 3, 4, 5, and 6 with the contents of four consecutive fullwords in memory
- The first fullword is at displacement 28 from base register 13
- The Assembler statement is:

LM 3,6,28(13)

RS instructions - LOAD MULTIPLE example

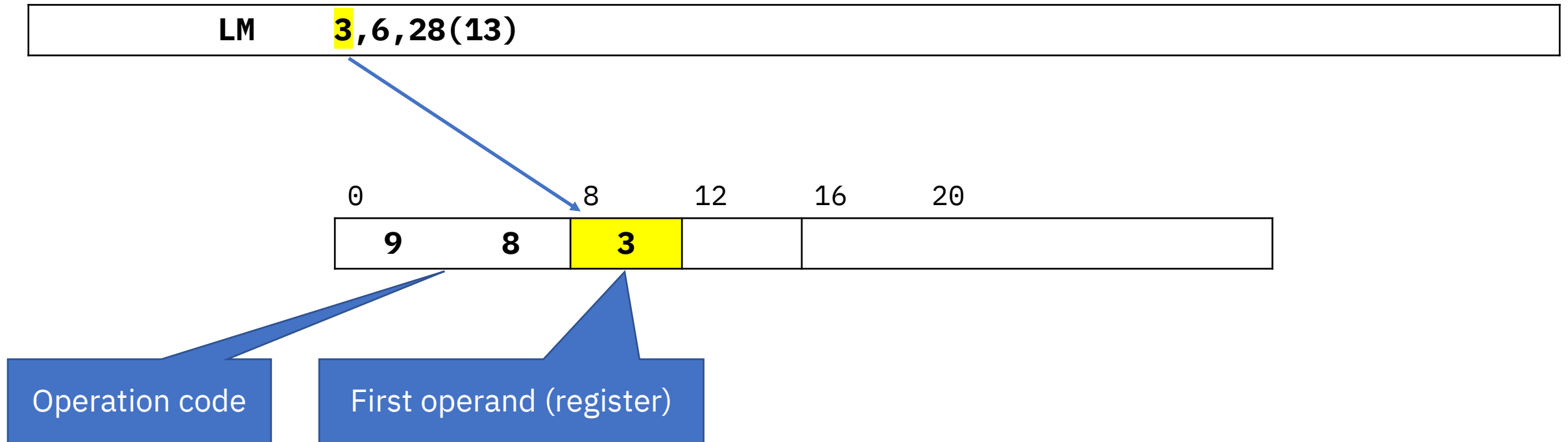
- The instruction, in memory, will be:



Operation code

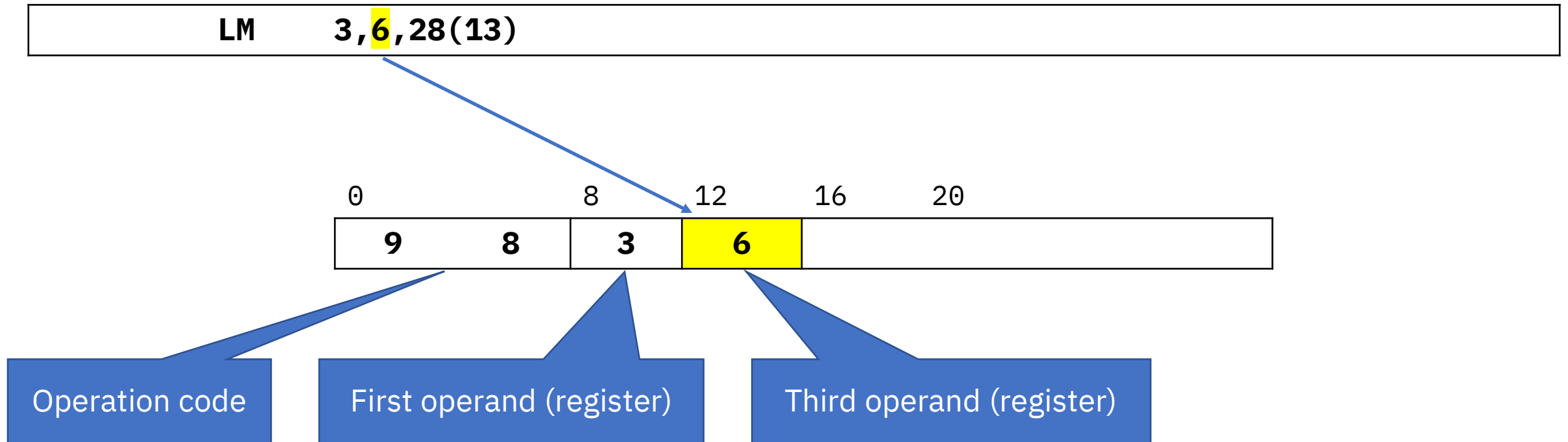
RS instructions - LOAD MULTIPLE example

- The instruction, in memory, will be:



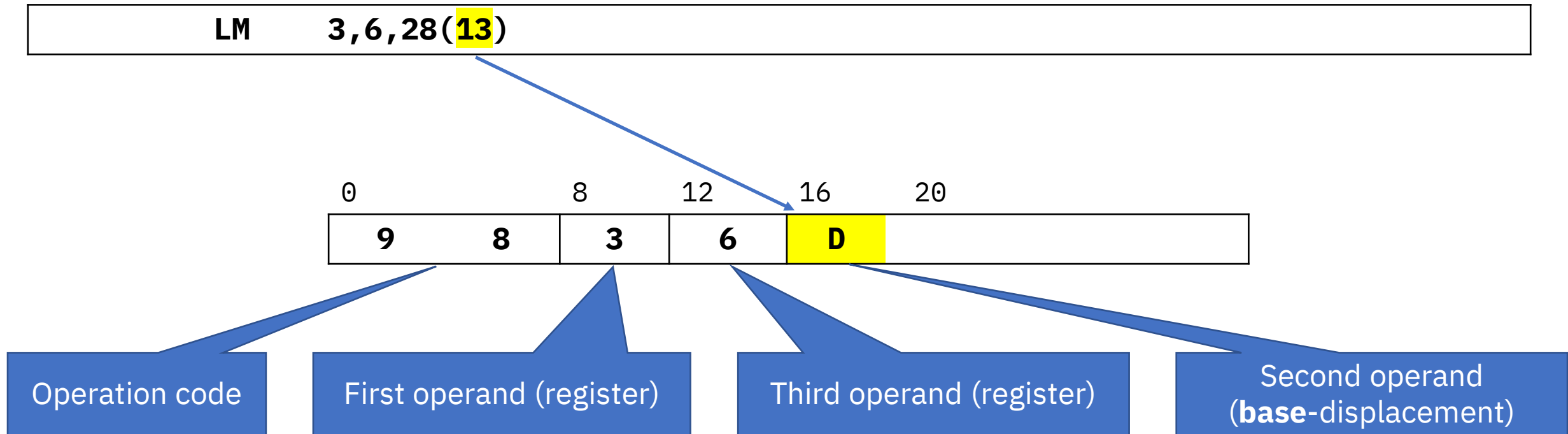
RS instructions - LOAD MULTIPLE example

- The instruction, in memory, will be:



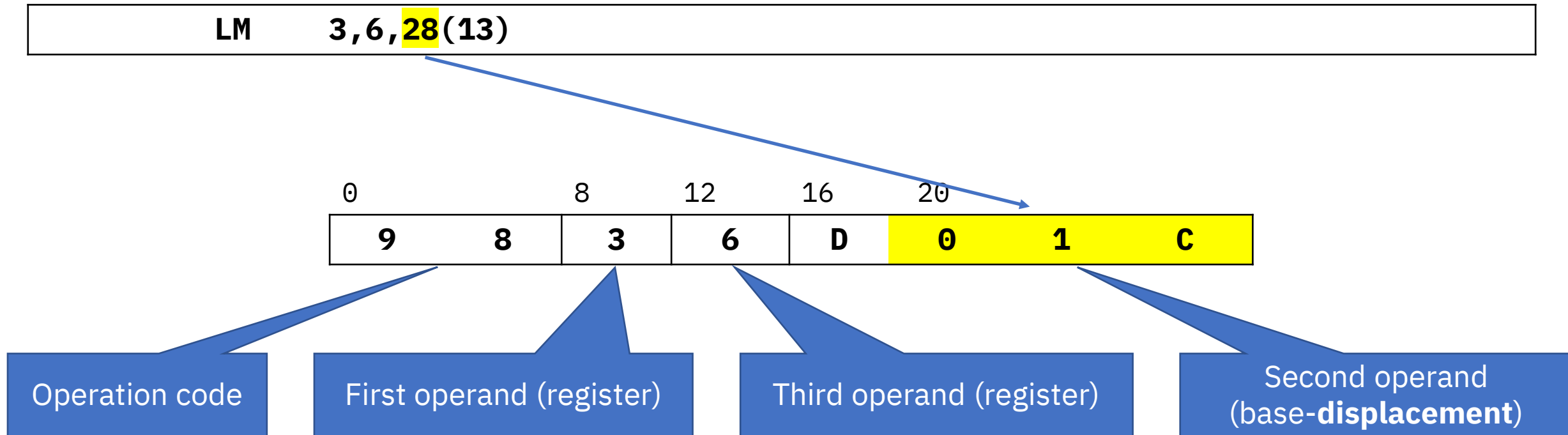
RS instructions - LOAD MULTIPLE example

- The instruction, in memory, will be:



RS instructions - LOAD MULTIPLE example

- The instruction, in memory, will be:



RS instructions - LOAD MULTIPLE example

- The instruction, in memory, will be:

LM	3,6,28(13)
-----------	-------------------

0	8	12	16	20			
9	8	3	6	D	0	1	C

Operation code

First operand (register)

Third operand (register)

Second operand
(base-displacement)

A brief note: it's easier than you think!

- Although it is possible to write a statement like this...

LM 3,6,28(13)

- ...you will most probably use names for storage areas
- For example, if register 13 points to a storage location called “SAVEAREA”, this statement is equivalent to the one above:

LM 3,6,SAVEAREA+28

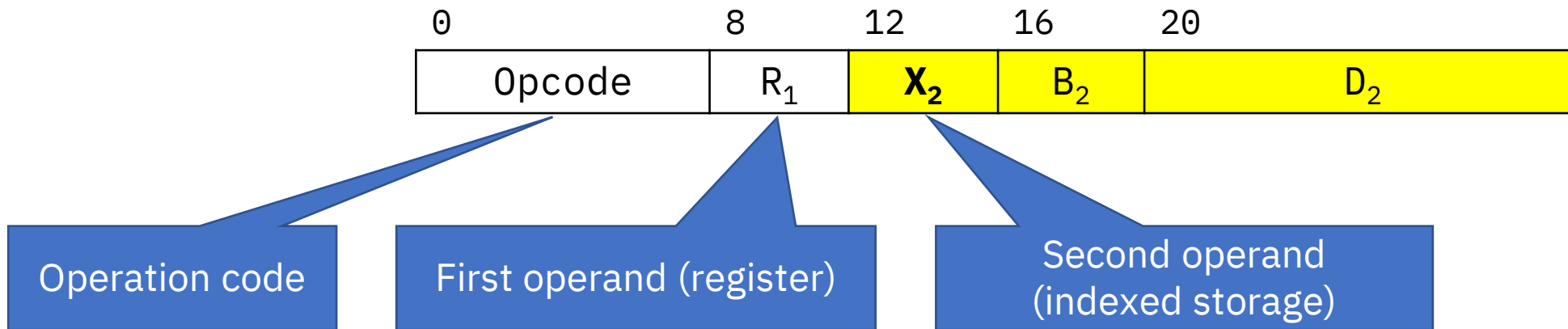
Instruction formats

- Next, RX instructions

Format	Description
Register-and-register (RR)	Both operands are registers
Register-and-storage (RS)	One operand is a register
Register-and-indexed-storage (RX)	and another a storage address
Storage-and-immediate (SI)	One operand is a storage address and another an “immediate” value
Storage-and-storage (SS)	Both operands are storage addresses
Register-and-storage (RXY)	One operand is a register and another a long displacement address
Register-and-immediate (RI)	One operand is a register and another a 16-bit relative address

RX instructions

- Indexed storage is a variation of base-displacement where another register, called the index, is added to form the effective address:



RX instructions - indexing

- With indexing, the effective address is **base + displacement + index**
- As in base-displacement, if the index is zero it means “no index”
- The Assembler syntax for indexed operands is:

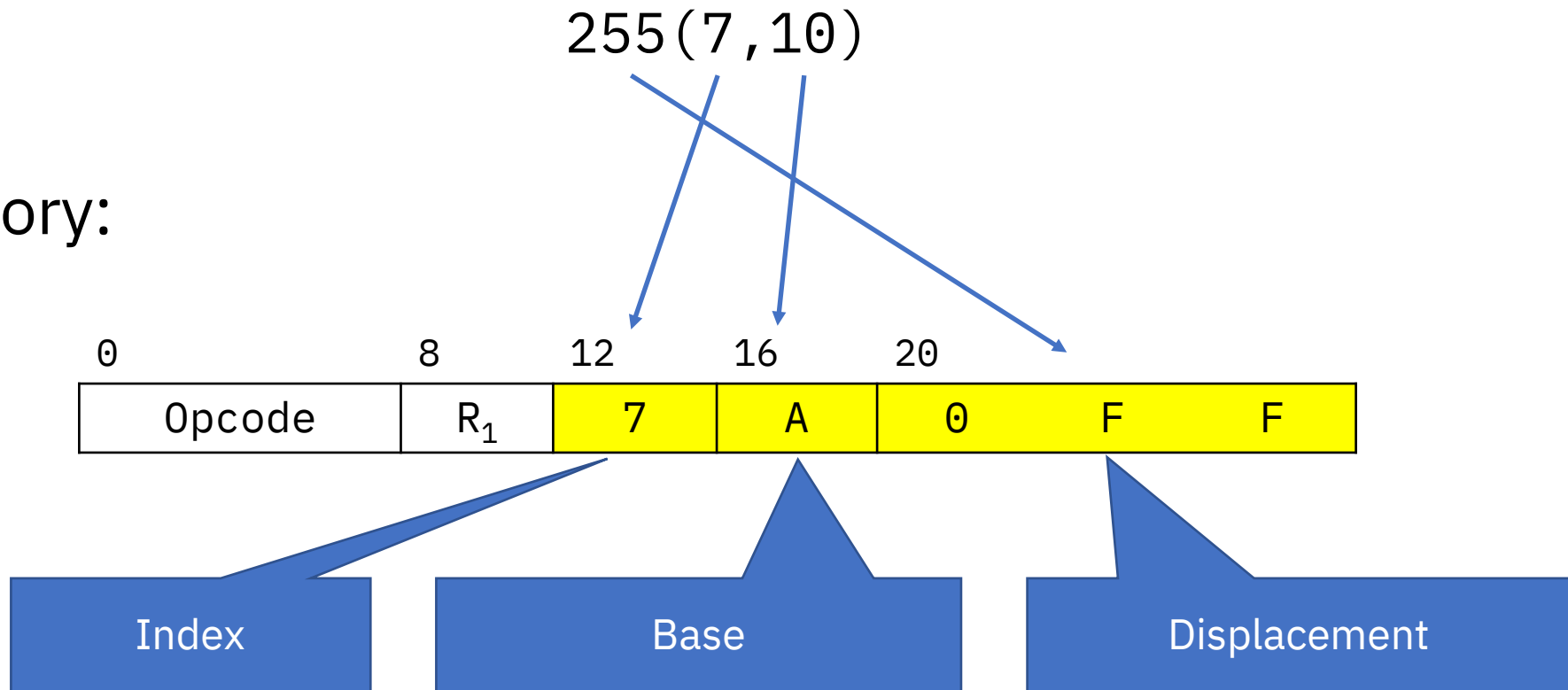
`displacement(index register, base register)`

- As the name implies, indexing can be used, for example, to address items in a table or array
 - The base register points to the array start; the index traverses the array

RX instructions - indexing

- Example: this operand address in Assembler...

- ...is, in memory:



RX instructions - ADD

- The ADD instruction adds a storage fullword to a register and puts the result in the register
 - Mnemonic **A**
 - Opcode **5A**
- Note this is different from the RR instruction ADD REGISTER we saw earlier
 - We'll see that most instructions come in variants of different formats

RX instructions - ADD example

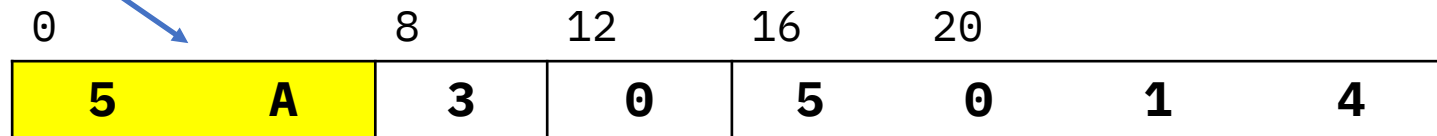
- “ADD to register 3 the fullword at displacement 20 from register 5”
- The Assembler source statement is:

A 3,20(0,5)

- Note in this case we are not using an index, so we specify zero

RX instructions - ADD example

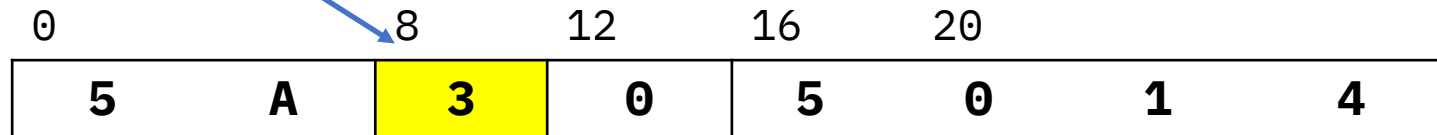
- The instruction, in memory, will be:



Operation code

RX instructions - ADD example

- The instruction, in memory, will be:



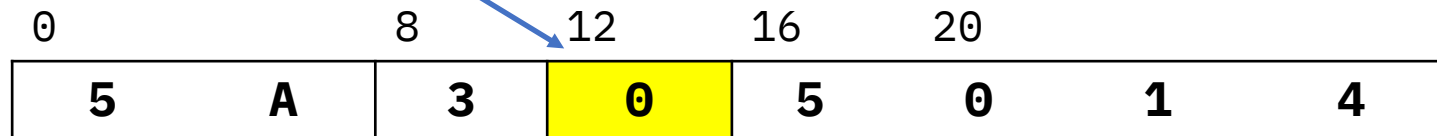
Operation code

First operand (register)

RX instructions - ADD example

- The instruction, in memory, will be:

A 3,20(0,5)



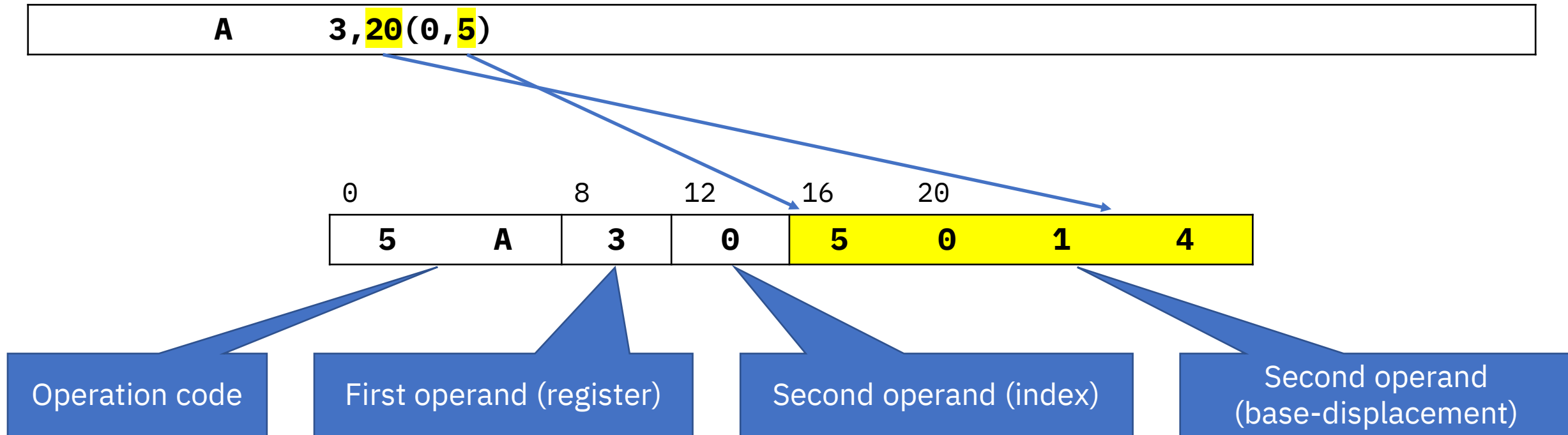
Operation code

First operand (register)

Second operand (index)

RX instructions - ADD example

- The instruction, in memory, will be:



Instruction formats

- Next, SI instructions

Format	Description
Register-and-register (RR)	Both operands are registers
Register-and-storage (RS)	One operand is a register
Register-and-indexed-storage (RX)	and another a storage address
Storage-and-immediate (SI)	One operand is a storage address and another an “immediate” value
Storage-and-storage (SS)	Both operands are storage addresses
Register-and-storage (RXY)	One operand is a register and another a long displacement address
Register-and-immediate (RI)	One operand is a register and another a 16-bit relative address